

P1272R3

Byteswapping for fun&&nuf

Published Proposal, 2020-07-14

This version:

<https://wg21.link/P1272R3>

Latest published version:

<https://wg21.link/P1272>

Author:

[Isabella Muerte](#)

Audience:

LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest source:

<src/byteswap.bs>

This source:

<src/byteswap.bs>

Implementation:

<slurps-mad-rips/byteswap>

Table of Contents

1 Revision History

- 1.1 Revision 3
- 1.2 Revision 2
- 1.3 Revision 1
- 1.4 Revision 0

2 Motivation

3 Design Considerations

3.1 Synopsis

4 Wording

4.1 Feature Testing

5 Acknowledgement

References

Informative References

§ 1. Revision History

§ 1.1. Revision 3

- Update wording to use C++20 features

§ 1.2. Revision 2

- Add missing remarks of undefined behavior in cases such as byteswapping an `IntegerType` such as a bitfield.
- Update feature macro value for the Cologne 2019 meeting

§ 1.3. Revision 1

- Update wording based on feedback from CWG regarding the representation of the "reversed" object.
- Update wording based on feedback from LWG subclause location and grammatical changes
- Readjust wording to use [\[N4820\]](#) library wording constructs instead of [\[N4672\]](#)

§ 1.4. Revision 0

Initial release. 🎉

§ 2. Motivation

Proposal [\[P0553r2\]](#) gives several bit operations to perform on integer types, such as popcount, or bit rotation. Despite these new operations provided to C++ developers, we still cannot swap (i.e., reverse) the bytes of builtin integer types in a performant way (i.e., one instruction or less) without resorting to compiler intrinsics. Currently, most CPU architectures provide single instructions for a byte swap. For those that don't, falling back on existing operations is more than amenable. We should, however, endeavor to standardize existing practice.

Note: The phrase one instruction or less refers to compilers inserting at most one instruction, and at the very least removing any instructions due to optimizations.

§ 3. Design Considerations

The design for the byteswap free function is quite simple. It takes any integer type and swaps its byteorder to the reverse of its current state. Additionally, it *only* takes integer types, requiring users to openly `bit_cast` their non-integers to integers in blatant heresy to the Worm-Seethe. How utterly disgraceful.

Note: It is intended that the byteswap function, despite swapping bytes, be placed into the `<bit>` header.

§ 3.1. Synopsis

The function's full specification is:

```
namespace std {  
    constexpr auto byteswap (integral auto value) noexcept;  
}
```

§ 4. Wording

The following synopsis is to be added to subclause 25.5.2 *Header* `<bit>` *Synopsis* [**bit.syn**]:

```
/ 25.5.4, byteswap  
constexpr auto byteswap (integral auto value) noexcept;  
  
...
```

The following is to be placed *before* the current subclause 25.5.4 [**bit.pow.two**] into [**bit.byteswap**]:

```
constexpr auto byteswap (integral auto value) noexcept;  
  
1 Mandates: value does not have padding bits.  
3 Returns: An object of the same type as value. Let the sequence R comprise the bytes of the  
object representation of value in reverse order. Each byte in the object representation of the result  
is equal to the byte in the corresponding position in R.
```

The following modification in 26.5.3 [**bit.cast**]:

```
template<typename To, typename From>  
    constexpr To bit_cast(const From& from) noexcept;
```

Returns: An object of type To. Each bit of the value representation of the result is equal to the corresponding bit in the object representation of from. Padding bits of the To object are unspecified. If there is no value of type To corresponding to the value representation produced, the behavior is undefined. If there are multiple such values, which value is produced is unspecified. A bit in the value representation of the result is indeterminate if does not correspond to a bit in the value representation of from or corresponds to a bit of an object that is not within its lifetime or has an indeterminate value ([basic.indet]). For each bit in the value representation of the result that is indeterminate, the smallest object containing that bit has an indeterminate value; the behavior is undefined unless that object is of unsigned ordinary character type or std::byte type. The result does not otherwise contain any indeterminate values.

§ 4.1. Feature Testing

In 16.3.1 [**support.limits.general**], Table 36, add a new row below `__cpp_lib_byte` with the following content:

Macro Name	Value	Header(s)
<code>__cpp_lib_byte</code>	201603L	<cstdint>
<code>__cpp_lib_byteswap</code>	<code>20????L</code>	<code><bit></code>
<code>__cpp_lib_char8_t</code>	201811L	<atomic> <filesystem> <istream> <limits> <locale> <ostream> <string> <string_view>

§ 5. Acknowledgement

Thanks to Nicole Muzzuca for feedback on writing this proposal.

§ References

§ Informative References

[N4672]

Casey Carter. [Editor's Report for the Ranges TS](https://wg21.link/n4672). 19 June 2017. URL: <https://wg21.link/n4672>

[N4820]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4820). 18 June 2019. URL: <https://wg21.link/n4820>

[P0553r2]

Jens Maurer. [Bit operations](https://wg21.link/p0553r2). 23 November 2017. URL: <https://wg21.link/p0553r2>